

Verification and Potential Security Issues in Microservices

Sahil Afrid Farookhi Mohammad
Department of Electrical and Computer Engineering
Iowa State University
Email: msaf@iastate.edu

Abstract—Microservices represent a distributed architectural pattern utilized for designing and developing services or APIs, facilitating secure communication between clients and servers. Despite the availability of several distributed architectures, microservices stand out for their scalability and loose coupling, enhancing resource management and utilization efficiency. However, as applications expand, various challenges arise that hinder effective communication between different services, potentially exposing vulnerabilities that malicious actors could exploit. This research aims to identify and address security issues inherent in microservices architecture, focusing on specific challenges such as authentication, authorization, and data integrity. It proposes a comprehensive model for verifying the architecture's robustness, leveraging advanced verification mechanisms to assess and enhance system integrity.

Innovatively, this study introduces a novel approach to addressing security concerns within a microservices architecture. By developing a suite of APIs that interact with the ChatGPT Service, the research integrates generative AI techniques to conduct a comprehensive security analysis. These APIs facilitate dynamic analysis of dependencies, perform static code analysis, conduct code reviews, and document code readability, providing a multi-dimensional assessment of the system's security posture. Moreover, leveraging AI enables not only the identification of vulnerabilities but also the generation of tailored solutions to enhance system resilience. By employing generative AI in the security analysis process, this research pioneers a proactive approach to microservices security, offering actionable insights to fortify distributed systems against emerging threats.

Index Terms – Distributed systems, microservices, security vulnerabilities, generative AI, code analysis, log analysis

I. INTRODUCTION

In the ever-evolving landscape of software development, the paradigm of distributed systems and microservices architecture [1] has emerged as a cornerstone for building scalable, flexible, and resilient applications. However, alongside the benefits offered by these architectures, the increasing complexity introduces a myriad of security challenges that demand innovative solutions. Understanding the intricate interplay between distributed systems, microservices, and security issues is imperative in ensuring the integrity and reliability of modern software applications.

Traditionally, the detection and mitigation of security vulnerabilities have relied on manual inspection, periodic code reviews, and adherence to coding standards. While these practices provide a foundation for maintaining code quality, they often fall short in addressing the dynamic and interconnected nature of modern software ecosystems. Recent events, such

as the Log4j vulnerability (Log4Shell), underscore the critical importance of proactive security measures in safeguarding against unforeseen vulnerabilities lurking within dependencies.

This research endeavors to bridge the gap between conventional security practices and the demands of contemporary software development by introducing a novel approach to security analysis. By leveraging the power of artificial intelligence, specifically Generative AI, this study aims to revolutionize the detection and remediation of security issues within distributed systems and microservices architectures. The integration of Generative AI with a suite of APIs developed using the Spring Boot framework enables comprehensive analysis of code, dependencies, and application logs.

Motivated by the imperative to enhance software security and mitigate the risks posed by evolving threat landscapes, this research seeks to make significant contributions to the field. By harnessing the capabilities of Generative AI, developers can expedite the process of identifying vulnerabilities, while also benefiting from actionable insights and recommended solutions. Ultimately, this research endeavors to empower developers to build more secure, robust, and resilient software applications in an era defined by rapid technological advancement and evolving security challenges.

A. Examples

Dependency Vulnerability: Equifax Data Breach [2]

In 2017, Equifax faced a major data breach due to an Apache Struts vulnerability (CVE-2017-5638), enabling attackers to execute code remotely and access the sensitive personal data of about 147 million consumers. Consequences included a \$575 million settlement with the US FTC, along with lawsuits and fines, damaging Equifax's reputation and consumer trust in data protection.

Bad Coding Standards: Target Data Breach [3]

In 2013, Target Corporation, a major US retailer, suffered a data breach due to vulnerabilities in its payment processing systems. Attackers exploited weak security protocols in a third-party vendor to access Target's network and exploited vulnerabilities in its custom payment software. Over 41 million customers' personal and financial data was compromised, leading to substantial financial losses, reputation damage, and legal liabilities. Target spent over \$200 million on remediation, fines, and settlements.

Improper Log Analysis: Hollywood Presbyterian Medical Center Ransomware Attack [4]

In 2016, Hollywood Presbyterian Medical Center in Los Angeles experienced a ransomware attack, encrypting patient records and disrupting operations. The attack exploited network vulnerabilities and phishing emails. Inadequate log analysis prolonged detection. The hospital declared an internal emergency, resorting to manual records, causing treatment delays. A \$17,000 Bitcoin ransom was paid, underscoring the financial and operational consequences of lax security and log analysis.

These real incidents illustrate the tangible consequences that businesses and organizations may face when vulnerabilities in dependencies, coding standards, and log analysis practices are exploited by malicious actors. They serve as cautionary tales and underscore the importance of robust cybersecurity [5] measures and proactive risk management strategies in safeguarding sensitive data and maintaining operational resilience.

II. METHODS

The primary objective of this research is to investigate how the integration of Generative AI [6] and microservices architecture can enhance security analysis in Spring Boot applications. Specifically, the research focuses on code analysis, dependency management, and log analysis to identify and mitigate security vulnerabilities within the application architecture.

The experimental setup employed in this research project utilized several key technologies and tools. Spring Boot, a popular Java-based framework, provided the foundation for developing the web applications under examination, offering features such as dependency injection and simplified configuration. Gradle, a powerful build automation tool, was utilized to manage project dependencies and streamline the build process. Java, a widely used programming language known for its portability and versatility, served as the primary language for coding the application logic. Postman, an API testing tool, played a crucial role in facilitating the interaction with the developed APIs, enabling comprehensive testing and validation. IntelliJ IDEA, an integrated development environment renowned for its robust features and user-friendly interface, served as the primary development environment for writing and debugging code. The experimental setup was executed on the MacOS operating system, chosen for its stability, developer-friendly ecosystem, and compatibility with the selected tools and technologies. This comprehensive setup ensured a conducive environment for conducting the experiments and analyzing the results effectively.

In the context of building APIs using Spring Boot within a microservices architecture, Spring Boot provides a powerful framework for rapidly developing and deploying microservices. APIs are designed and built using Spring Boot by leveraging its features such as Spring MVC for building RESTful APIs, Spring Data for data access, and Spring Security for implementing security measures. To design APIs with Spring Boot, developers typically define API endpoints

using annotations like `@RestController` to mark a class as a controller and `@RequestMapping` to specify the URL mapping for each endpoint. Data transfer objects (DTOs) are often used to define the structure of data exchanged between the client and server. Spring Boot simplifies API development by providing auto-configuration, which reduces the need for manual configuration and boilerplate code. Additionally, Spring Boot's embedded application server allows APIs to be packaged as standalone JAR files, making deployment straightforward. Furthermore, Spring Boot integrates seamlessly with other Spring projects such as Spring Cloud for building distributed systems and managing microservices. This integration facilitates tasks like service discovery, load balancing, and fault tolerance in a microservices environment. In summary, Spring Boot offers a robust framework for designing and building APIs within a microservices architecture, enabling developers to create scalable, resilient, and maintainable systems.

A. Spring Boot Architecture with Swagger UI and Dependencies:

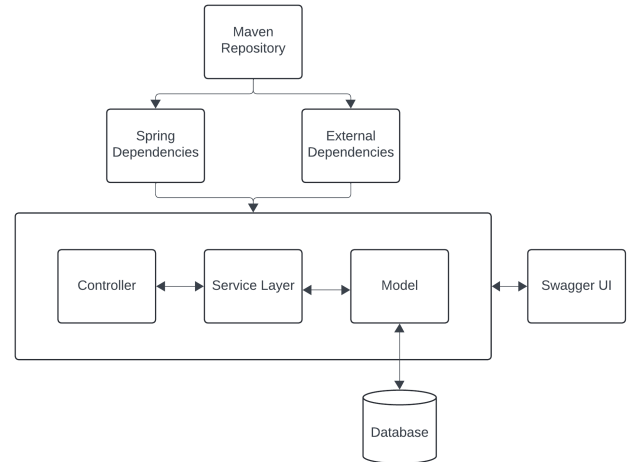


Fig. 1. Architecture of a Spring Boot Application with Maven Dependencies and Swagger UI

Exploring the intricate communication dynamics within the Microservices architecture, particularly employing Spring Boot, unveils a structured flow of interactions among its pivotal components as shown in Figure 1.

The communication paradigm commences with the controller's reception and routing of incoming requests, subsequently instigating interactions cascading through the service layer, model, and database. The presence of Swagger UI not only enriches developers' understanding of exposed APIs but also augments the collaborative development endeavor.

B. Proposed Solution Architecture

In dissecting the flow depicted within the block diagram of the proposed research project grounded in the Microservices architecture, each component's role and interaction are delineated as follows in Figure 2.

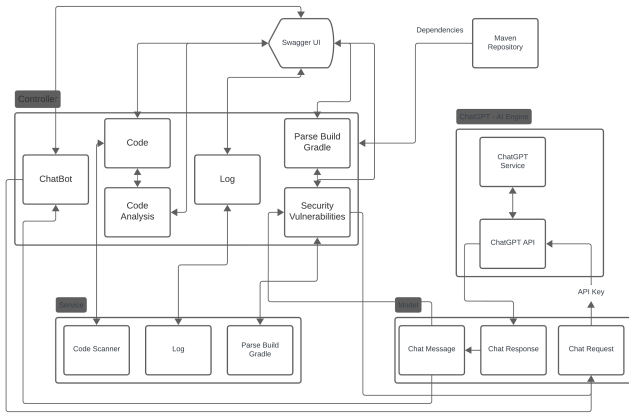


Fig. 2. Proposed Architecture Diagram of Spring Boot Application Integrating with ChatGPT Service via API Key

1) *Controller*: Serving as the inaugural node in the flow, the Controller assumes the pivotal task of intercepting and processing incoming requests. These requests, ranging from code analysis to log assessment or dependency scrutiny, initiate the system's operational sequence. Functioning as the interface between external systems or users and the application, the Controller serves as the gateway for interactions.

2) *ChatBot*: Enmeshed within the architecture, the ChatBot emerges as a central entity, contributing significantly to code analysis endeavors. It engages in parsing chat messages, extracting pertinent code segments or files to facilitate subsequent analysis procedures.

3) *Code Analysis*: Transitions into the domain of Code Analysis, where the extracted code snippets or files undergo meticulous scrutiny. This phase entails a comprehensive examination to unearth security vulnerabilities, potential bugs, and adherence to coding standards.

4) *Log Analysis*: In parallel, the Controller orchestrates the dispatch of logs or log-related inquiries to the Log Analysis module. Within this sphere, logs generated by the application undergo systematic review to detect anomalies, errors, or suspicious activities, thereby facilitating the identification of security threats or performance impediments.

5) *Dependencies Analysis*: Anchoring the application's operational framework are its inherent Dependencies, which undergo meticulous scrutiny. Interaction with the Dependencies component is facilitated by the Controller, with data sourced from the Maven Repository. Scrutiny encompasses the detection of security vulnerabilities, compatibility assessments, and perusal of licensing constraints. Notably, the application's Swagger UI may also play a role in this analytical endeavor.

6) *Security Vulnerabilities*: Converging outcomes from Code Analysis and Dependencies Analysis converge to identify and delineate security vulnerabilities. These vulnerabilities encompass a spectrum of concerns, ranging from the presence of insecure libraries to outdated dependencies, necessitating targeted remedial interventions to fortify system resilience.

7) *ChatGPT Service*: Culminating in the ChatGPT Service [7], processed data comprising code analysis insights, log diagnostics, and dependency profiles are transmitted. Authentication and authorization are facilitated through a Secret API key [8], orchestrating access to the ChatGPT model. Interactions with the ChatGPT model engender further insights, recommendations, or responses, underpinned by the contextual analysis of the transmitted data.

8) *Chat Message and Response*: Fostering bidirectional communication, users or external systems transmit chat messages to the ChatGPT Service. The Service, in turn, processes these messages, potentially leveraging the analyzed data to contextualize responses. Responses, generated by the ChatGPT model, are relayed back to the originating user or system, completing the interaction loop.

C. Leveraging ChatGPT Service for Proactive Security Enhancement

The research implements a sophisticated architecture to leverage the ChatGPT Service for analyzing logs, project dependencies, and code components, aiming to identify and address security vulnerabilities within a Spring Boot application. This section delineates the core components and functionalities of the architecture, emphasizing their roles in detecting and mitigating potential security threats through the integration of the ChatGPT Service.

1) *Architecture Overview*: The architecture encompasses several pivotal components, including controllers, services, and model classes, each contributing to the overarching goal of enhancing security within the application. These components collaborate seamlessly to enable the detection and resolution of security vulnerabilities across multiple facets of the system, including log management, project dependency analysis, and code parsing.

2) Component Descriptions:

a) *Log Analysis Component*: Central to the architecture is the LogController, which exposes endpoints for retrieving and analyzing application logs. Leveraging the LogService, the /analyze endpoint orchestrates log analysis by constructing a prompt for the ChatGPT Service. Upon receiving the analyzed response, the architecture facilitates appropriate action based on identified security vulnerabilities. The integration of the ChatGPT Service empowers the system to detect nuanced security threats within log data, enabling proactive measures to mitigate potential risks.

b) *Project Dependency Analysis Component*: The architecture incorporates the ParseBuildGradleController and SecurityVulnerabilitiesController to manage the parsing and analysis of project dependencies for security vulnerabilities. These controllers interact with the ParseBuildGradle service to extract dependency details from the "build.gradle" file and formulate prompts for the ChatGPT Service. By leveraging the insights provided by the ChatGPT Service, the system can identify and address security vulnerabilities inherent in project dependencies, thereby bolstering the application's resilience against external threats.

c) *Code Analysis Component*: The architecture includes controllers such as CodeController and CodeAnalysisController to facilitate the analysis of code components for security vulnerabilities. These controllers collaborate with the CodeScannerService to retrieve code snippets and interact with the ChatGPT Service for analysis. By harnessing the capabilities of the ChatGPT Service, the system can detect and remediate security vulnerabilities embedded within the codebase, fortifying the application’s defenses against potential exploits and breaches.

The implemented architecture represents a robust framework for leveraging the ChatGPT Service to proactively identify and mitigate security vulnerabilities within a Spring Boot application. By integrating the ChatGPT Service into various components such as log analysis, project dependency analysis, and code parsing, the system can effectively detect, analyze, and address security threats across multiple dimensions. This proactive approach to security empowers organizations to safeguard their applications against evolving threats, ensuring the integrity, confidentiality, and availability of critical systems and data.

III. RESULTS AND DISCUSSION

A. Accessing Endpoints in Spring Boot Using Swagger UI or Postman:

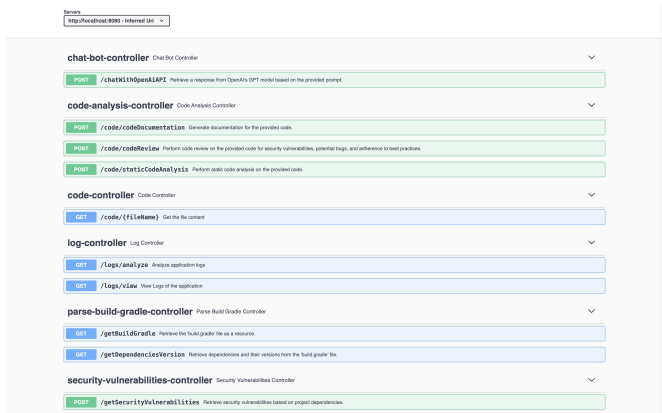


Fig. 3. Swagger UI Displaying API Endpoints

After launching your Spring Boot application, you can easily interact with its endpoints using Swagger UI or Postman. Swagger UI is a widely-used tool for visualizing and testing RESTful APIs. It generates interactive documentation based on the OpenAPI Specification, offering a user-friendly interface to explore endpoints. Access Swagger UI by navigating to `http://localhost:8080/swagger-ui.html` if your application is running on port 8080. You’ll find a comprehensive list of endpoints with descriptions, parameters, and example payloads. Swagger UI allows executing API requests from the browser and viewing real-time responses, aiding development and testing.

On the other hand, Postman provides a versatile environment for creating, testing, and automating HTTP requests.

To interact with your Spring Boot endpoints using Postman, launch the Postman application and create a new request collection or workspace. Then, specify the HTTP method, endpoint URL, request headers, parameters, and payload as needed. Postman excels in handling more advanced testing scenarios, including scripting, collections, and environments. It offers a comprehensive solution for testing different aspects of your API, such as authentication, headers, and response validation.

B. Security Vulnerabilities:

From Figure 4, the findings are as follows:

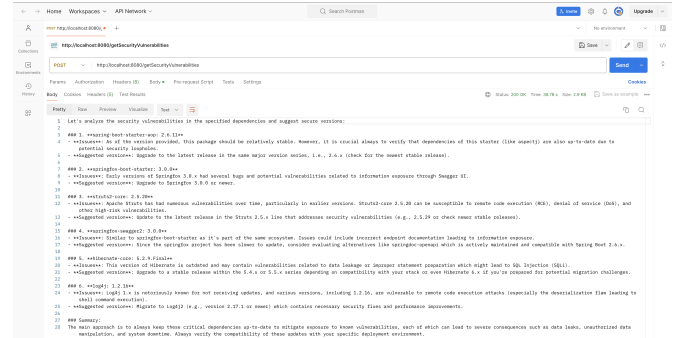


Fig. 4. Security Vulnerabilities Detected in Application via Postman

a) *spring-boot-starter-aop*: In the case of ‘spring-boot-starter-aop 2.6.11’, although free from CVE-2022-22965 (Spring4Shell), continuous vigilance is advised with migration towards the latest releases. Continued use without migration may expose systems to potential future vulnerabilities, necessitating ongoing monitoring and proactive updates.

b) *springfox-boot-starter*: For ‘springfox-boot-starter 3.0.0’, while direct vulnerabilities remain scant, upgrading to version 3.0.1 or newer is recommended for stability and compatibility enhancements. Failure to upgrade may result in compatibility issues and missed stability enhancements, warranting a transition to version 3.0.1 or newer for optimal performance.

c) *log4j2*: Lastly, ‘log4j 1.2.16’ poses substantial risks due to its discontinued maintenance and susceptibility to CVE-2017-5645, warranting an upgrade to log4j2 version 2.17.1 or newer. Persistent use of ‘log4j 1.2.16’ increases exposure to known vulnerabilities, mandating an immediate transition to log4j2 version 2.17.1 or later to mitigate potential security breaches.

C. Log Analysis:

The analysis of logs from the Spring Boot application reveals several important insights from Figure 5:

1) *Log Structure*: Each entry includes a timestamp, log level, thread information, logger, and a message describing the event.

2) *Patterns Identified*:

a) *Repetition of Actions*: There’s a recurring pattern of requesting and parsing the build.gradle file, which could indicate either routine tasks or potential inefficiencies.

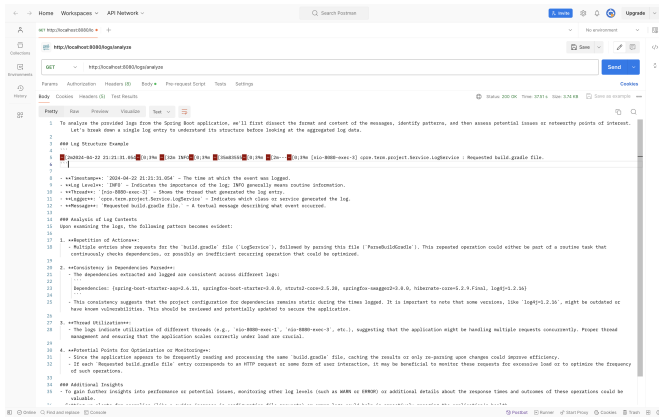


Fig. 5. Postman Displaying Application Log Analysis

b) Consistency in Dependencies: The dependencies extracted from the logs remain static, suggesting a stable project configuration. However, outdated dependencies like `log4j=1.2.16` pose security risks.

c) Thread Utilization: Various threads are utilized, indicating concurrent request handling. Proper thread management and scalability are important.

d) Optimization Opportunities: Caching results or optimizing the frequency of file parsing could improve performance.

3) Insights for Optimization and Monitoring: Monitoring other log levels and response times could provide further insights into performance and potential issues. Setting up alerts for anomalies or errors could help in proactive management.

While log entries indicate normal operations, there are opportunities for performance enhancement (such as caching) and security improvement (updating dependencies). Continual monitoring, along with broader observability of system health, is crucial for maintaining a robust application.

D. Code Analysis:

The provided results encompass a thorough analysis of a Java codebase focused on parsing files within a Spring-based application. The evaluation is divided into three main sections: Code Review, Static Code Analysis [9], and Code Documentation, as shown in Figures 6, 7, and 8 respectively.

1) Code Review:

a) Security Vulnerabilities and Potential Bugs: Identified vulnerabilities such as path traversal, resource handling concerns, inefficient regex pattern compilation, and potential logic errors.

b) Adherence to Best Practices: Highlighted concerns related to resource management in Spring, logging sensitive information, code separation, deprecated API usage, and regex usage for parsing.

c) Recommendations: Offered actionable suggestions to refactor code, improve resource handling, enhance validation and error handling, consider alternatives to regex, and review logging strategies.

2) Static Code Analysis:

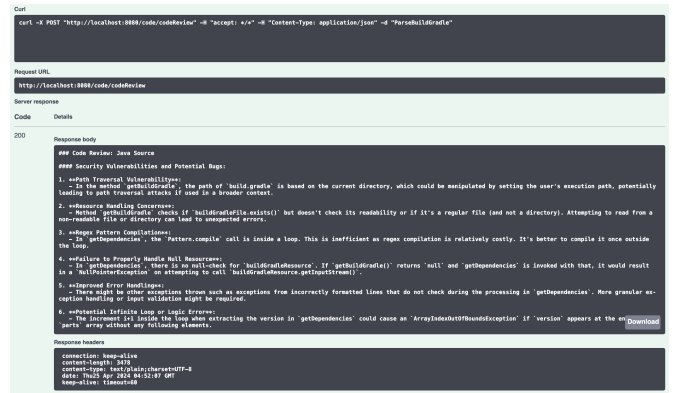


Fig. 6. Code Review Security Vulnerabilities and Good Practices Adherence Results in Swagger UI

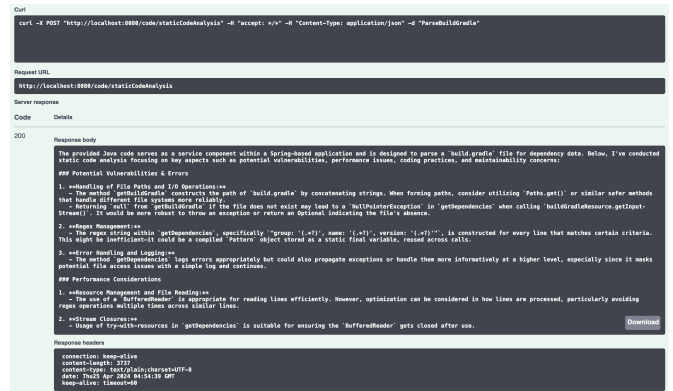


Fig. 7. Static Code Analysis Results Displayed in Swagger UI

a) Potential Vulnerabilities & Errors: Detailed potential issues regarding file path handling, regex management, error handling, and logging.

b) Performance Considerations: Examined resource management, file reading efficiency, and stream closures for optimal performance.

c) Coding Practices: Evaluated code clarity, maintainability, and dependency extraction logic and provided recommendations for improvement.

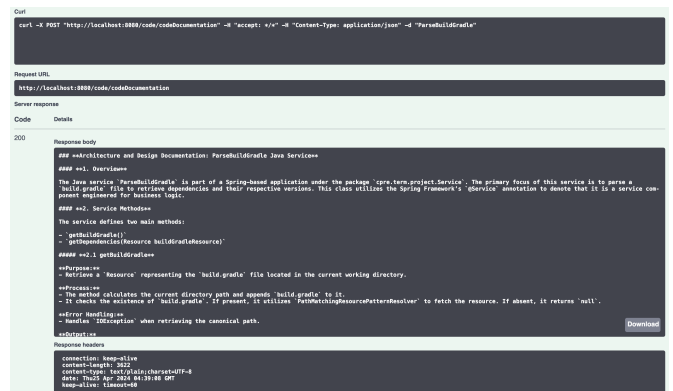


Fig. 8. Swagger UI Display of the Code Documentation

3) Code Documentation:

a) *Architecture and Design Documentation:* Provided an overview of the Java service, its methods, purpose, process, error handling, utilized components, logging, exception handling, future enhancements, and dependencies.

b) *Diagrams:* Mentioned the potential use of UML Sequence Diagrams and Class Diagrams for visualizing interactions and structures.

This comprehensive analysis presents a holistic view of the codebase, addressing security, performance, coding practices, and documentation aspects, laying a foundation for further improvements and optimizations.

This new approach will help to make the application better. Traditional practices might not be as effective as AI engines (ChatGPT) in finding security vulnerabilities and issues. ChatGPT analyzes application dependencies, code, and logs. Existing approaches only address minor predefined issues during static code analysis. Log management tools send alerts if any issues arise. Codebase hosting platforms such as GitHub only send the security report after the code is pushed and the entire repository is scanned. Manual code reviews are not as efficient as AI, which offers a broader perspective. Documentation tools lack readability because they generate documents based on predefined rules. The introduced solution can handle many aspects thanks to the intelligence from the ChatGPT service. ChatGPT has a wide range of information not only to analyze and report but also to suggest workarounds or fixes for the issues.

IV. CONCLUSION

In conclusion, the proposed approach presents a significant advancement in enhancing application security. By leveraging AI engines such as ChatGPT, our methodology surpasses traditional practices in identifying security vulnerabilities and issues. Unlike existing methods, which primarily focus on predefined issues through static code analysis or rely on log management tools for issue detection, our approach comprehensively analyzes application dependencies, code structures, and logs.

Moreover, conventional practices like manual code reviews and documentation tools are inherently limited in scope and efficiency compared to AI-driven solutions. Manual reviews lack the breadth and depth provided by AI systems, while documentation tools often produce documents that are rigid and lack readability.

The integration of ChatGPT intelligence enables our solution to address various aspects of application security comprehensively. ChatGPT not only analyzes and reports vulnerabilities but also offers suggestions for effective workarounds or fixes. This holistic approach ensures a more robust and proactive security posture for applications.

Looking ahead, the future scope of this system is promising. It can be further developed into a plugin or dependency that can be hosted on the Maven repository, making it easily accessible and integrable into various software projects. Additionally, by leveraging cloud technologies, the system can

be enhanced to achieve production-grade quality and support different environments such as local, testing, development, and production. This step would make the entire software portable and readily usable by any application, further extending its utility and impact in enhancing application security.

Integrating generative AI techniques into microservices architecture for security analysis presents innovative possibilities, but it also brings potential drawbacks. These include resource intensiveness, reliance on quality training data, risks of bias in AI models, privacy concerns, and the need for continuous maintenance to adapt to evolving threats. Addressing these challenges is crucial for ensuring the successful implementation and effectiveness of the approach in strengthening distributed systems against emerging security threats.

In summary, this research demonstrates the efficacy of employing AI engines like ChatGPT Service to enhance application security, offering a versatile solution capable of addressing diverse security challenges comprehensively while also paving the way for future advancements and integration possibilities.

ACKNOWLEDGEMENTS

I express my sincere gratitude to Professor Dr. Yong Guan for his invaluable guidance, support, and mentorship throughout my research project. His expertise, continuous encouragement, and insightful feedback significantly enhanced the quality and success of my work. I am deeply grateful for his unwavering commitment to my academic development.

REFERENCES

- [1] P. D. Francesco, I. Malavolta, and P. Lago, "Research on architecting microservices: Trends, focus, and potential for industrial adoption," in *2017 IEEE International Conference on Software Architecture (ICSA)*, 2017, pp. 21–30.
- [2] Federal Trade Commission, "Equifax data breach settlement," Federal Trade Commission, December 2022. [Online]. Available: <https://www.ftc.gov/enforcement/refunds/equifax-data-breach-settlement>
- [3] Committee on Commerce, Science, and Transportation, "A 'kill chain' analysis of the 2013 target data breach," Available: <https://www.commerce.senate.gov/services/files/24d3c229-4f2f-405d-b8db-a3a67f183883>, Mar. 2014.
- [4] S. T. Argaw, N.-E. Bempong, B. Eshaya-Chauvin, and A. Flahault, "The state of research on cyberattacks against hospitals and available best practice recommendations: a scoping review," *BMC Medical Informatics and Decision Making*, vol. 19, no. 1, Jan. 2019. [Online]. Available: <https://doi.org/10.1186/s12911-018-0724-5>
- [5] D. Kant and A. Johannsen, "Evaluation of ai-based use cases for enhancing the cyber security defense of small and medium-sized companies (smes)," *Electronic Imaging*, vol. 34, pp. 1–8, 2022.
- [6] G. Rani, J. Singh, and A. Khanna, "Comparative analysis of generative ai models," in *2023 International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)*, 2023, pp. 760–765.
- [7] S. Rice, S. R. Crouse, S. R. Winter, and C. Rice, "The advantages and limitations of using chatgpt to enhance technological research," *Technology in Society*, vol. 76, p. 102426, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0160791X23002312>
- [8] C. M. Gallardo Paredes, C. Machuca, and Y. M. Semblantes Claudio, "Chatgpt api: Brief overview and integration in software development," *International Journal of Engineering Insights*, vol. 1, no. 1, pp. 25–29, Nov. 2023. [Online]. Available: <https://injei.scithoth.com/index.php/journal/article/view/7>
- [9] I. Gomes, P. Morgado, T. Gomes, and R. Moreira, "An overview on the static code analysis approach in software development," *Faculdade de Engenharia da Universidade do Porto, Portugal*, vol. 16, 2009.